

Итоговый проект. Разработка и развертывание ИИ-приложения в Kubernetes

Дисциплина: Оркестрация и контейнеризация

Проект (экзамен): Разработка и развёртывание ИИ-приложения в Kubernetes

Студент: Березняк В.Н., гр. М24-535

Среда выполнения: Ubuntu 22.04.5 LTS (VMware Workstation) + VS Code (Windows 11, Remote-SSH)

Дата: 29.12.2025

Структура репозитория

```
.
├── Report. Development and deployment AI in k8s.pdf
├── README.md
├── app/
│   ├── SentimentApplication.java
│   └── Dockerfile
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── hpa.yaml
│   └── service-monitor.yaml
└── screenshots/
    ├── 01_vm/
    ├── 02_remote_ssh/
    ├── 03_minikube/
    ├── 04_docker/
    ├── 05_k8s/
    ├── 06_monitoring/
    └── 07_schemes/
```

1. Введение

1.1. Цель и задачи проекта

Цель проекта — продемонстрировать полный цикл разработки, контейнеризации и развертывания Java-приложения с простым анализом тональности текста в локальном кластере Kubernetes (Minikube), а также показать базовую интеграцию с системой мониторинга Prometheus + Grafana.

Задачи:

1. Разработать простое Java-приложение с REST API:
 - эндпоинт `GET /api/sentiment?text=...`, возвращающий JSON с оценкой тональности;
 - эндпоинт `GET /health` для проверки здоровья;
 - эндпоинт `GET /metrics` в формате Prometheus для сбора метрик.
2. Упаковать приложение в Docker-образ размером менее 150 МБ (multi-stage build).
3. Развернуть приложение в Kubernetes (Minikube) с:
 - Deployment минимум на 3 реплики;
 - Service типа `LoadBalancer`;
 - Ingress для маршрутизации запросов на `/api` и `/health`;

- HPA (Horizontal Pod Autoscaler) по CPU.
- 4. Настроить мониторинг:
 - установка Prometheus + Grafana через Helm ([kube-prometheus-stack](#));
 - подключение приложения через [ServiceMonitor](#).
- 5. Подготовить раздел анализа тенденций по статьям arXiv.

1.2. Используемый технологический стек

Компонент	Технология
Язык/Runtime	Java SE 17
REST	com.sun.net.httpserver.HttpServer
Контейнеризация	Docker
Оркестрация	Kubernetes (Minikube)
Управление	kubectl
Мониторинг	Prometheus + Grafana (kube-prometheus-stack)
Балансировка	Service (LoadBalancer) + Ingress
Масштабирование	HPA по CPU

1.3. Среда выполнения

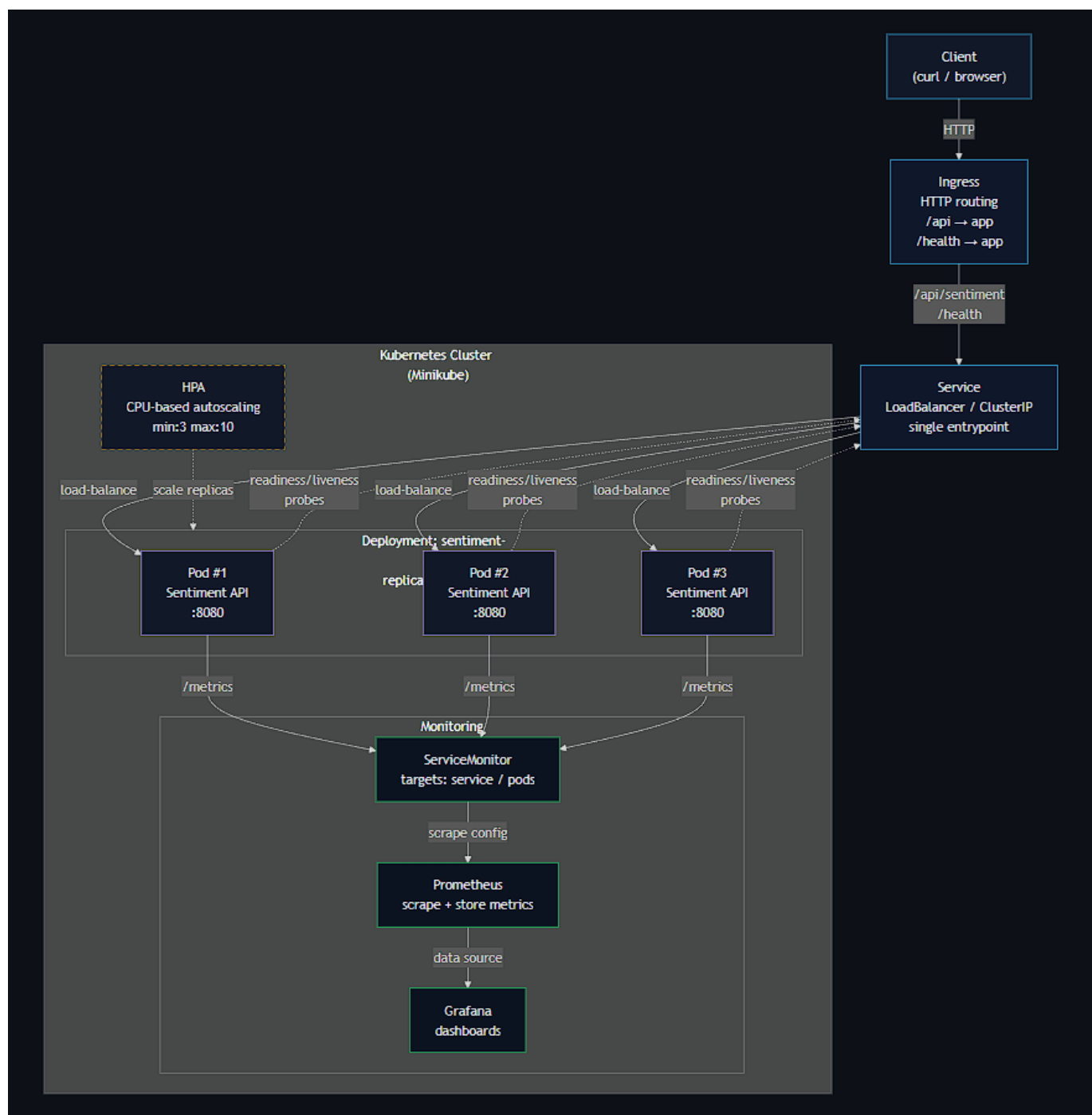
- Ubuntu 22.04.5 LTS (виртуальная машина в VMware Workstation). См. в папке [screenshots/01_vm](#)
- На хосте Windows 11: VS Code + расширения Remote Development / Remote-SSH. См. в папке [screenshots/02_remote_ssh](#)
- Минимальные ресурсы VM (рекомендация): 4 vCPU, 8+ GB RAM, 20+ GB диск

2. Архитектура решения

2.1. Логическая схема

Компоненты решения:

1. **Sentiment API (Java-приложение)**
 - [/api/sentiment](#) — анализ тональности;
 - [/health](#) — health-проверка;
 - [/metrics](#) — метрики в формате Prometheus.
2. **Kubernetes Deployment**
 - 3 реплики Pod'ов с контейнером [sentiment-app](#);
 - probes (liveness/readiness), requests/limits.
3. **Service (LoadBalancer)**
 - единая точка входа и балансировка по репликам.
4. **Ingress**
 - маршрутизация HTTP на [/api](#) и [/health](#).
5. **HPA**
 - автоскейлинг по CPU (3...10 реплик).
6. **Prometheus + Grafana**
 - сбор метрик Kubernetes и приложения через [ServiceMonitor](#);
 - визуализация в Grafana.



3. Развёртывание инфраструктуры Minikube

3.1. Установка Minikube и kubectl

3.1.1. Установка Minikube

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

```
minikube version
```

```

Ubuntu 64-bit x
bereza@k8s-ubuntu:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

sudo install minikube % Total      % Received % Xferd  Average Sp-peed  Time      Time      Time  Current
t
          Dload  Upload    Total    Spent    Left  Speed
   0    0    0    0    0    0      0     0  --:--:--  --:--:--  --:--:--    0linux-amd64 /usr/local
100 133M 100 133M    0    0 1632k      0  0:01:23  0:01:23  --:--:-- 1763k
bereza@k8s-ubuntu:~$
bereza@k8s-ubuntu:~$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
[sudo] password for bereza:
bereza@k8s-ubuntu:~$ minikube version
minikube version: v1.37.0
commit: 65318f4cfff9c12cc87ec9eb8f4cdd57b25047f3
bereza@k8s-ubuntu:~$ _

```

3.1.2. Установка kubectl

```

curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

kubectl version --client

```

```

Ubuntu 64-bit x
bereza@k8s-ubuntu:~$ curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable
.txt)/bin/linux/amd64/kubectl"
% Total      % Received % Xferd  Average Speed  Time      Time      Time  Current
          Dload  Upload    Total    Spent    Left  Speed
100   138 100   138    0    0   393      0  --:--:--  --:--:--  --:--:--   394
100 55.8M 100 55.8M    0    0 6583k      0  0:00:08  0:00:08  --:--:-- 6789k
bereza@k8s-ubuntu:~$ sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
bereza@k8s-ubuntu:~$ kubectl version --client
Client Version: v1.35.0
Kustomize Version: v5.7.1
bereza@k8s-ubuntu:~$

```

3.2. Создание кластера Minikube

Запуск кластера:

```
minikube start --cpus=4 --memory=6144mb --nodes=2
```

```

berez@k8s-ubuntu:~$ minikube start --driver=docker --cpus=4 --memory=8192mb --nodes=2
🐳 minikube v1.37.0 on Ubuntu 22.04
🔧 Using the docker driver based on user configuration
🔧 Using Docker driver with root privileges
🔥 Starting "minikube" primary control-plane node in "minikube" cluster
📡 Pulling base image v0.0.48 ...
🔧 Creating docker container (CPUs=4, Memory=8192MB) ...
🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
🔗 Configuring CNI (Container Networking Interface) ...
🔧 Verifying Kubernetes components...
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass

🔥 Starting "minikube-m02" worker node in "minikube" cluster
📡 Pulling base image v0.0.48 ...
🔧 Creating docker container (CPUs=4, Memory=8192MB) ...
🌐 Found network options:
   ▪ NO_PROXY=192.168.49.2
🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
   ▪ env NO_PROXY=192.168.49.2
🔧 Verifying Kubernetes components...
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
berez@k8s-ubuntu:~$

```

Проверка статуса, нодов и подов:

```

minikube status
kubectl get nodes -o wide
kubectl get pods -A

```

```

berez@k8s-ubuntu:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

minikube-m02
type: Worker
host: Running
kubelet: Running

berez@k8s-ubuntu:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION        CONTAINER-RUNTIME
minikube      Ready     control-plane   13m   v1.34.0   192.168.49.2   <none>        Ubuntu 22.04.5 LTS   5.15.0-164-generic    docker://28.4.0
minikube-m02  Ready     <none>        12m   v1.34.0   192.168.49.3   <none>        Ubuntu 22.04.5 LTS   5.15.0-164-generic    docker://28.4.0

berez@k8s-ubuntu:~$ kubectl get pods -A
NAMESPACE     NAME                                     READY   STATUS    RESTARTS   AGE
kube-system   coredns-66bc5c9577-ffwb4               1/1     Running   2 (12m ago)  13m
kube-system   etcd-minikube                          1/1     Running   0           13m
kube-system   kindnet-qwf56                          1/1     Running   0           12m
kube-system   kindnet-zpqfj                          1/1     Running   0           13m
kube-system   kube-apiserver-minikube                 1/1     Running   0           13m
kube-system   kube-controller-manager-minikube        1/1     Running   0           13m
kube-system   kube-proxy-6p44d                        1/1     Running   0           13m
kube-system   kube-proxy-gj4bq                        1/1     Running   0           12m
kube-system   kube-scheduler-minikube                 1/1     Running   0           13m
kube-system   storage-provisioner                     1/1     Running   1 (12m ago)  13m

```

3.3. Включение необходимых аддонов

Для работы Ingress и HPA включаю аддоны `ingress` и `metrics-server`:

```

minikube addons enable ingress
minikube addons enable metrics-server

minikube addons list | egrep 'ingress|metrics-server'

```

```

berez@k8s-ubuntu:~$ minikube addons enable ingress
⚠ ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
  ▪ Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
🔍 Verifying ingress addon...
🌟 The 'ingress' addon is enabled
berez@k8s-ubuntu:~$ minikube addons enable metrics-server
⚠ metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  ▪ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.0
🌟 The 'metrics-server' addon is enabled
berez@k8s-ubuntu:~$ minikube addons list | egrep 'ingress|metrics-server'
  ingress      minikube  enabled  ✓  Kubernetes
  ingress-dns   minikube  disabled
  metrics-server minikube  enabled  ✓  Kubernetes
berez@k8s-ubuntu:~$

```

Для доступа к сервисам типа **LoadBalancer** использую:

```
minikube tunnel
```

minikube tunnel нужно держать запущенным в отдельном терминале.

```

berez@k8s-ubuntu: ~
berez@k8s-ubuntu:~$ minikube tunnel
[sudo] password for berez:
Status:
  machine: minikube
  pid: 21998
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: []
errors:
  minikube: no errors
  router: no errors
  loadbalancer emulator: no errors

```

4. Контейнеризация Java-приложения

4.1. Реализация REST-API анализа тональности

Приложение написано на Java SE 17 (без Spring), через `com.sun.net.httpserver.HttpServer`.

Эндпоинты:

- `GET /api/sentiment?text=...` → JSON с тональностью и score
- `GET /health` → `{"status": "UP"}`
- `GET /metrics` → метрика `sentiment_requests_total` (Prometheus text format)

Код приложения находится в файле:

- `app/SentimentApplication.java`

4.2. Dockerfile и multi-stage build

Dockerfile с multi-stage сборкой и `jlink` для минимального JRE:

- `app/Dockerfile`

4.3. Сборка и локальное тестирование образа

Сборка образа:

```
cd final-k8s-sentiment-template/app
docker build -t sentiment-app:1.0 .
docker images sentiment-app:1.0
```

Убедимся, что размер <150 MB

```
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ docker build -t sentiment-app:1.0 .
[+] Building 49.5s (16/16) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 774B                                0.0s
=> [internal] load metadata for docker.io/library/alpine:3.20     3.7s
=> [internal] load metadata for docker.io/library/eclipse-temurin:17-jdk-alpine 3.7s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [builder 1/5] FROM docker.io/library/eclipse-temurin:17-jdk-alpine@sha256:eaf56b7430cee6c93871106367715e2675192093d8f67d 27.5s
=> => resolve docker.io/library/eclipse-temurin:17-jdk-alpine@sha256:eaf56b7430cee6c93871106367715e2675192093d8f67dbbde0713 0.0s
=> => sha256:f03fcdf37379fff8027b1eaf41f78daa4bf57cd4f423a9e9d7015d3bbcf9bb53 2.28kB / 2.28kB 0.6s
=> => sha256:b3851e26ddd2575c157cfd1ad2a673f8e8b4541dd5f6a0b8a2f65d3935e10fe6 129B / 129B 1.0s
=> => sha256:ecc9308f1b3f4d73b7c1a1aaba0b8eda9e9155caf3619e236d0c1a8b410704ef 143.99MB / 143.99MB 19.1s
=> => sha256:2da2b7fa39d9ca89504884bfa364de57e68f656d1fd46f04941d3d34ffdbca30 21.11MB / 21.11MB 8.6s
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB 3.4s
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.4s
=> => extracting sha256:2da2b7fa39d9ca89504884bfa364de57e68f656d1fd46f04941d3d34ffdbca30 1.7s
=> => extracting sha256:ecc9308f1b3f4d73b7c1a1aaba0b8eda9e9155caf3619e236d0c1a8b410704ef 8.1s
=> => extracting sha256:b3851e26ddd2575c157cfd1ad2a673f8e8b4541dd5f6a0b8a2f65d3935e10fe6 0.0s
=> => extracting sha256:f03fcdf37379fff8027b1eaf41f78daa4bf57cd4f423a9e9d7015d3bbcf9bb53 0.0s
=> [internal] load build context                                    0.1s
=> => transferring context: 4.16kB                                    0.0s
=> [stage-1 1/5] FROM docker.io/library/alpine:3.20@sha256:765942a4039992336de8dd5db680586e1a206607dd06170ff0a37267a9e01958 1.8s
=> => resolve docker.io/library/alpine:3.20@sha256:765942a4039992336de8dd5db680586e1a206607dd06170ff0a37267a9e01958 0.0s
=> => sha256:5311e7f182d02360a7194aa2995849bcdff04795c39a0ffdcf413eae625865970 3.63MB / 3.63MB 1.3s
=> => extracting sha256:5311e7f182d02360a7194aa2995849bcdff04795c39a0ffdcf413eae625865970 0.3s
=> [stage-1 2/5] WORKDIR /app                                       0.1s
=> [builder 2/5] WORKDIR /app                                       4.1s
=> [builder 3/5] COPY SentimentApplication.java .                  0.0s
=> [builder 4/5] RUN javac SentimentApplication.java && jar cfe app.jar SentimentApplication SentimentApplication.class 3.2s
=> [builder 5/5] RUN jlink --module-path /opt/java/openjdk/jmods --add-modules java.base,java.logging,jdk.httpserver 6.3s
=> [stage-1 3/5] COPY --from=builder /opt/jre-min /opt/jre-min     0.2s
=> [stage-1 4/5] COPY --from=builder /app/app.jar /app/app.jar     0.0s
=> [stage-1 5/5] RUN addgroup -S appgroup && adduser -S appuser -G appgroup && chown -R appuser:appgroup /app 0.5s
=> exporting to image                                              3.3s
=> => exporting layers                                              2.7s
=> => exporting manifest sha256:90dd3d3c650f9a2569395b56e2e9a7cf3a48d438e6c017884a85571c0936b4d3 0.0s
=> => exporting config sha256:122ec3e779c8537a590378eea78e9366359b46593e7f3e30d877c1098a02023c 0.0s
=> => exporting attestation manifest sha256:d5064ba65b36cf2d356bd5ad1545ffd36ed9f3fe952168eac1c8064b9ba71f08 0.0s
=> => exporting manifest list sha256:c9dad30a1e1f01b66e124508208e81a476a9780e22388d7b3d2adb29afb233bb 0.0s
=> => naming to docker.io/library/sentiment-app:1.0              0.0s
=> => unpacking to docker.io/library/sentiment-app:1.0          0.5s
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ docker images sentiment-app:1.0
Info → U In Use
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
sentiment-app:1.0	c9dad30a1e1f	72.8MB	25.4MB	

```
○ bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$
```

Локальный запуск:

```
docker run --rm -p 8080:8080 --name sentiment-test sentiment-app:1.0
```

Проверка эндпоинтов (в другом терминале):

```
curl "http://localhost:8080/api/sentiment?text=I+love+Kubernetes"
curl "http://localhost:8080/health"
curl "http://localhost:8080/metrics"
```

```

berez@k8s-ubuntu:~/final-k8s-sentiment-template/app$ docker run --rm -p 8080:8080 \
--name sentiment-test sentiment-app:1.0
SentimentApplication started on port 8080
[]

berez@k8s-ubuntu:~$ curl "http://localhost:8080/api/sentiment?text=I+love+Kubernetes"
{"text":"I love Kubernetes","sentiment":"positive","score":0.85}berez@k8s-ubuntu:~$
berez@k8s-ubuntu:~$ curl "http://localhost:8080/health"
{"status":"UP"}berez@k8s-ubuntu:~$
berez@k8s-ubuntu:~$ curl "http://localhost:8080/metrics"
# HELP sentiment_requests_total Total number of sentiment requests
# TYPE sentiment_requests_total counter
sentiment_requests_total 6
berez@k8s-ubuntu:~$

```

4.4. Загрузка образа в Minikube (multi-node)

Кластер Minikube запущен с 2 нодами, поэтому `minikube docker-env` не применяется.

Для доставки образа в кластер используется команда `minikube image load`.

```

cd ~/final-k8s-sentiment-template/app
minikube image load sentiment-app:1.0
minikube image ls | grep sentiment-app

```

```

berez@k8s-ubuntu:~$ cd ~/final-k8s-sentiment-template/app
berez@k8s-ubuntu:~/final-k8s-sentiment-template/app$ minikube image load sentiment-app:1.0
berez@k8s-ubuntu:~/final-k8s-sentiment-template/app$ minikube image ls | grep sentiment-app
docker.io/library/sentiment-app:1.0
berez@k8s-ubuntu:~/final-k8s-sentiment-template/app$

```

5. Развёртывание приложения в Kubernetes

5.1. Kubernetes-манифесты

Файлы лежат в папке `k8s/`:

1. `k8s/deployment.yaml` — Deployment (3 реплики) + ServiceAccount.
2. `k8s/service.yaml` — Service LoadBalancer.
3. `k8s/ingress.yaml` — Ingress.
4. `k8s/hpa.yaml` — HPA по CPU.
5. `k8s/service-monitor.yaml` — ServiceMonitor для Prometheus.

5.2. Применение манифестов

```

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml
kubectl apply -f k8s/hpa.yaml

kubectl get deployments
kubectl get pods -o wide
kubectl get svc
kubectl get ingress
kubectl get hpa

```



```

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/ingress.yaml
kubectl apply -f k8s/hpa.yaml
deployment.apps/sentiment-app created
serviceaccount/sentiment-sa created
service/sentiment-service created
ingress.networking.k8s.io/sentiment-ingress created
horizontalpodautoscaler.autoscaling/sentiment-hpa created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl get deployments
kubectl get pods -o wide
kubectl get svc
kubectl get ingress
kubectl get hpa
NAME          READY  UP-TO-DATE  AVAILABLE  AGE
sentiment-app 3/3      3           3           20s
NAME          READY  STATUS    RESTARTS  AGE  IP          NODE          NOMINATED NODE  READINESS GATES
sentiment-app-58684d8c86-ctk9z 1/1    Running   0          20s  10.244.1.10 minikube-m02  <none>          <none>
sentiment-app-58684d8c86-fqkzd 1/1    Running   0          20s  10.244.0.8   minikube      <none>          <none>
sentiment-app-58684d8c86-h5zvz 1/1    Running   0          20s  10.244.1.11 minikube-m02  <none>          <none>
NAME          TYPE          CLUSTER-IP  EXTERNAL-IP  PORT(S)          AGE
kubernetes    ClusterIP     10.96.0.1   <none>       443/TCP          129m
sentiment-service LoadBalancer 10.105.164.4 <pending>     80:30657/TCP    20s
NAME          CLASS  HOSTS          ADDRESS          PORTS  AGE
sentiment-ingress nginx  sentiment.local 192.168.49.2     80     21s
NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
sentiment-hpa Deployment/sentiment-app cpu: <unknown>/50% 3          10        3          20s
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$

```

5.3. Тестирование доступа к приложению

5.3.1. Через port-forward

```

kubectl port-forward svc/sentiment-service 8080:80
curl "http://localhost:8080/api/sentiment?text=I+like+Kubernetes"

```

```

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl port-forward svc/sentiment-service 8080:80
Forwarding from 127.0.0.1:8080 -> 8080
Forwarding from [::1]:8080 -> 8080
Handling connection for 8080
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ curl "http://localhost:8080/api/sentiment?text=I+like+Kubernetes"
{"text":"I like Kubernetes","sentiment":"positive","score":0.85}
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$

```

5.3.2. Через Ingress

После настройки /etc/hosts и запуска minikube tunnel:

```

curl "http://sentiment.local/api/sentiment?text=hello"
curl "http://sentiment.local/health"

```

```

● bereza@k8s-ubuntu:~$ minikube tunnel
[sudo] password for bereza:
Status:
  machine: minikube
  pid: 71222
  route: 10.96.0.0/12 -> 192.168.49.2
  minikube: Running
  services: [sentiment-service]
errors:
  minikube: no errors
  router: no errors
  loadbalancer emulator: no errors
● bereza@k8s-ubuntu:~$ curl "http://sentiment.local/api/sentiment?text=hello"
{"text":"hello","sentiment":"neutral","score":0.85}
● bereza@k8s-ubuntu:~$ curl "http://sentiment.local/health"
{"status":"UP"}
● bereza@k8s-ubuntu:~$

```

5.4. Проверка работы HPA

Создаю нагрузку:

```
cd ~/final-k8s-sentiment-template/app
kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
```

```
● bereza@k8s-ubuntu:~$ cd ~/final-k8s-sentiment-template/app
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl run load-generator --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
pod/load-generator created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl run load-generator-2 --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
pod/load-generator-2 created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl run load-generator-3 --image=busybox --restart=Never -- /bin/sh -c 'while true; do wget -q -O- http://sentiment-service/api/sentiment?text=load; done'
pod/load-generator-3 created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template/app$ kubectl get hpa sentiment-hpa
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
sentiment-hpa	Deployment/sentiment-app	cpu: 46%/50%	3	10	4	28m

Слежение за масштабированием:

```
kubectl get hpa -w
kubectl get pods -w
```

```
● bereza@k8s-ubuntu:~$ kubectl get hpa sentiment-hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
sentiment-hpa	Deployment/sentiment-app	cpu: 32%/50%	3	10	3	22m
sentiment-hpa	Deployment/sentiment-app	cpu: 30%/50%	3	10	3	23m
sentiment-hpa	Deployment/sentiment-app	cpu: 35%/50%	3	10	3	24m
sentiment-hpa	Deployment/sentiment-app	cpu: 59%/50%	3	10	3	25m

```
● bereza@k8s-ubuntu:~$ kubectl get pods -l app=sentiment-app -w
```

NAME	READY	STATUS	RESTARTS	AGE
sentiment-app-58684d8c86-ctk9z	1/1	Running	0	23m
sentiment-app-58684d8c86-fqkzd	1/1	Running	0	23m
sentiment-app-58684d8c86-h5zvz	1/1	Running	0	23m
sentiment-app-58684d8c86-jvkvb	0/1	Pending	0	0s
sentiment-app-58684d8c86-jvkvb	0/1	Pending	0	0s
sentiment-app-58684d8c86-jvkvb	0/1	ContainerCreating	0	1s
sentiment-app-58684d8c86-jvkvb	0/1	Running	0	3s
sentiment-app-58684d8c86-jvkvb	1/1	Running	0	11s

6. Мониторинг: Prometheus и Grafana

6.1. Установка Helm и kube-prometheus-stack

6.1.1. Установка Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
```

```
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 11929  100 11929    0     0  30600      0 --:--:-- --:--:-- --:--:-- 30665
Downloading https://get.helm.sh/helm-v3.19.4-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
[sudo] password for bereza:
helm installed into /usr/local/bin/helm
version.BuildInfo{Version:"v3.19.4", GitCommit:"7c6b6e486dac026202556836bb910c37d847793e", GitTreeState:"clean", GoVersion:"go1.24.11"}
○ bereza@k8s-ubuntu:~/final-k8s-sentiment-template$
```

6.1.2. Установка стека мониторинга

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

kubectl create namespace monitoring

helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --
set prometheus.prometheusSpec.serviceMonitorSelectorNilUsesHelmValues=false --set
grafana.adminPassword=admin123 --wait
```

```

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100 11929  100 11929    0     0  30600    0 --:--:-- --:--:-- --:--:-- 30665
Downloading https://get.helm.sh/helm-v3.19.4-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
[sudo] password for bereza:
helm installed into /usr/local/bin/helm
version.BuildInfo{Version:"v3.19.4", GitCommit:"7cfeb6486dac026202556836bb010c37d847793e", GitTreeState:"clean", GoVersion:"go1.24.11"}
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
"prometheus-community" has been added to your repositories
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ♪Happy Helming!♪
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl create namespace monitoring
namespace/monitoring created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ helm install prometheus prometheus-community/kube-prometheus-stack --namespace monitoring --set prometheus.prometheus
Spec.serviceMonitorSelectorNilUsesHelmValues=false --set grafana.adminPassword=admin123 --wait
I1229 13:06:35.720127 288270 warnings.go:110] "Warning: unrecognized format \"int32\""
I1229 13:06:35.720201 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:35.962503 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:35.962565 288270 warnings.go:110] "Warning: unrecognized format \"int32\""
I1229 13:06:36.046618 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:36.046673 288270 warnings.go:110] "Warning: unrecognized format \"int32\""
I1229 13:06:36.096067 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:36.905772 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:36.905842 288270 warnings.go:110] "Warning: unrecognized format \"int32\""
I1229 13:06:37.582125 288270 warnings.go:110] "Warning: unrecognized format \"int32\""
I1229 13:06:37.582232 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:37.714635 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:38.275733 288270 warnings.go:110] "Warning: unrecognized format \"int32\""
I1229 13:06:38.275870 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:38.340000 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:38.693326 288270 warnings.go:110] "Warning: unrecognized format \"int32\""
I1229 13:06:38.693385 288270 warnings.go:110] "Warning: unrecognized format \"int64\""
I1229 13:06:53.503761 288270 warnings.go:110] "Warning: spec.SessionAffinity is ignored for headless services"
I1229 13:06:53.503748 288270 warnings.go:110] "Warning: spec.SessionAffinity is ignored for headless services"
I1229 13:06:53.504561 288270 warnings.go:110] "Warning: spec.SessionAffinity is ignored for headless services"
I1229 13:06:53.505219 288270 warnings.go:110] "Warning: spec.SessionAffinity is ignored for headless services"
I1229 13:06:53.505281 288270 warnings.go:110] "Warning: spec.SessionAffinity is ignored for headless services"
NAME: prometheus
LAST DEPLOYED: Mon Dec 29 13:06:41 2025
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=prometheus"

Get Grafana 'admin' user password by running:

  kubectl --namespace monitoring get secrets prometheus-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo

Access Grafana local instance:

  export POD_NAME=$(kubectl --namespace monitoring get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=prometheus" -oname)
  kubectl --namespace monitoring port-forward $POD_NAME 3000

Get your grafana admin user password by running:

  kubectl get secret --namespace monitoring -l app.kubernetes.io/component=admin-secret -o jsonpath="{.items[0].data.admin-password}" | base64 --decode ; echo

Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$

```

Проверка:

```

helm list -n monitoring
kubectl get pods -n monitoring
kubectl get svc -n monitoring

```

```

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ helm list -n monitoring
NAME      NAMESPACE    REVISION    UPDATED                               STATUS    CHART              APP VERSION
prometheus monitoring    1           2025-12-29 13:06:41.153439556 +0000 UTC deployed  kube-prometheus-stack-80.8.0  v0.87.1

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-prometheus-kube-prometheus-alertmanager-0  2/2     Running   0           7m
prometheus-grafana-68b8bc859-r965g                      3/3     Running   0           7m13s
prometheus-kube-prometheus-operator-5fcf4f6f45-vpl6w     1/1     Running   0           7m13s
prometheus-kube-state-metrics-66f9f5bf55-vlzqd           1/1     Running   0           7m13s
prometheus-prometheus-kube-prometheus-prometheus-0      2/2     Running   0           7m
prometheus-prometheus-node-exporter-qk6ff                1/1     Running   0           7m13s
prometheus-prometheus-node-exporter-w99wn                1/1     Running   0           7m13s

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl get svc -n monitoring
NAME                                TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
alertmanager-operated              ClusterIP  None         <none>        9093/TCP,9094/TCP,9094/UDP  7m18s
prometheus-grafana                 ClusterIP  10.109.21.70 <none>        80/TCP           7m30s
prometheus-kube-prometheus-alertmanager ClusterIP  10.96.252.91 <none>        9093/TCP,8080/TCP  7m30s
prometheus-kube-prometheus-operator ClusterIP  10.102.108.27 <none>        443/TCP          7m30s
prometheus-kube-prometheus-prometheus ClusterIP  10.102.134.115 <none>        9090/TCP,8080/TCP  7m30s
prometheus-kube-state-metrics      ClusterIP  10.108.157.240 <none>        8080/TCP         7m30s
prometheus-operated                ClusterIP  None         <none>        9090/TCP         7m17s
prometheus-prometheus-node-exporter ClusterIP  10.99.219.96 <none>        9100/TCP         7m30s

```

6.2. Подключение приложения к Prometheus (ServiceMonitor)

Применяю **ServiceMonitor**:

```

kubectl apply -f k8s/service-monitor.yaml
kubectl get servicemonitor -A

```

```

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl apply -f k8s/service-monitor.yaml
servicemonitor.monitoring.coreos.com/sentiment-app-monitor created
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl get servicemonitor -A
NAMESPACE   NAME                                AGE
default     sentiment-app-monitor              8s
monitoring  prometheus-grafana                11m
monitoring  prometheus-kube-prometheus-alertmanager 11m
monitoring  prometheus-kube-prometheus-apiserver   11m
monitoring  prometheus-kube-prometheus-coredns     11m
monitoring  prometheus-kube-prometheus-kube-controller-manager 11m
monitoring  prometheus-kube-prometheus-kube-etcd   11m
monitoring  prometheus-kube-prometheus-kube-proxy  11m
monitoring  prometheus-kube-prometheus-kube-scheduler 11m
monitoring  prometheus-kube-prometheus-kubelet     11m
monitoring  prometheus-kube-prometheus-operator    11m
monitoring  prometheus-kube-prometheus-prometheus  11m
monitoring  prometheus-kube-state-metrics          11m
monitoring  prometheus-prometheus-node-exporter     11m

```

6.3. Доступ к Prometheus и Grafana

```

kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090 &
kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80 &

```

```

● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl port-forward -n monitoring svc/prometheus-kube-prometheus-prometheus 9090:9090 &
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ kubectl port-forward -n monitoring svc/prometheus-grafana 3000:80 &
[1] 295594
[2] 295595
● bereza@k8s-ubuntu:~/final-k8s-sentiment-template$ Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::1]:9090 -> 9090
Forwarding from [::1]:3000 -> 3000

```

Так как port-forward выполняется внутри Ubuntu-BM, для открытия интерфейсов с хостовой Windows нужен SSH-туннель:

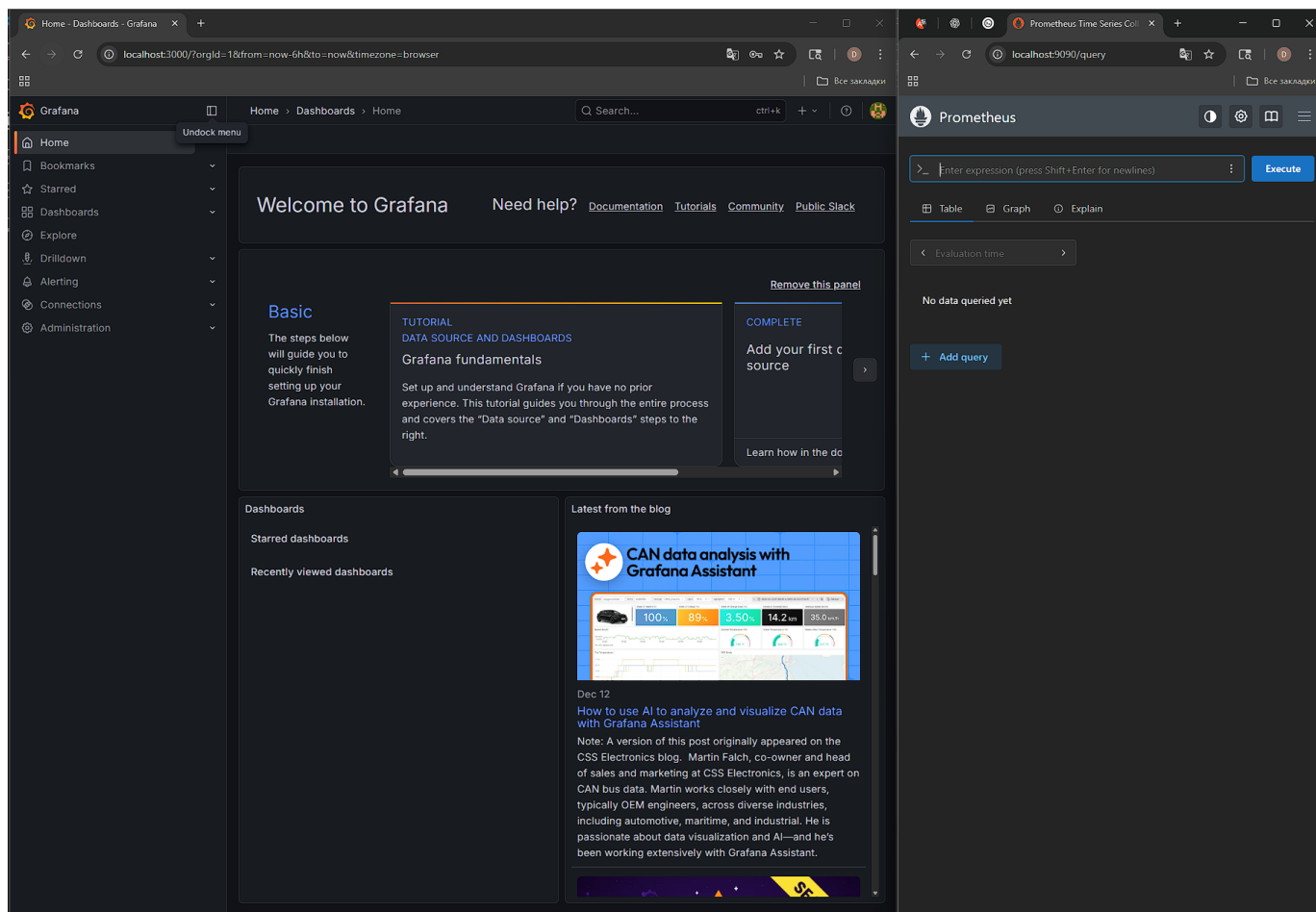
```

ssh -L 3000:127.0.0.1:3000 -L 9090:127.0.0.1:9090 bereza@192.168.112.128

```

```
bereza@k8s-ubuntu: ~  
Windows PowerShell  
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.  
  
Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows  
  
PS C:\Users\Владимир> ssh -L 3000:127.0.0.1:3000 -L 9090:127.0.0.1:9090 bereza@192.168.112.128  
bind [127.0.0.1]:3000: Permission denied  
bind [127.0.0.1]:9090: Permission denied  
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-164-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Mon Dec 29 01:30:26 PM UTC 2025  
  
System load:  0.67           Processes:            386  
Usage of /:   46.7% of 34.15GB Users logged in:       1  
Memory usage: 36%           IPv4 address for ens33: 192.168.112.128  
Swap usage:   0%  
  
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s  
  just raised the bar for easy, resilient and secure K8s cluster deployment.  
  
  https://ubuntu.com/engage/secure-kubernetes-at-the-edge  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
New release '24.04.3 LTS' available.  
Run 'do-release-upgrade' to upgrade to it.  
  
Last login: Mon Dec 29 10:02:51 2025  
bereza@k8s-ubuntu:~$
```

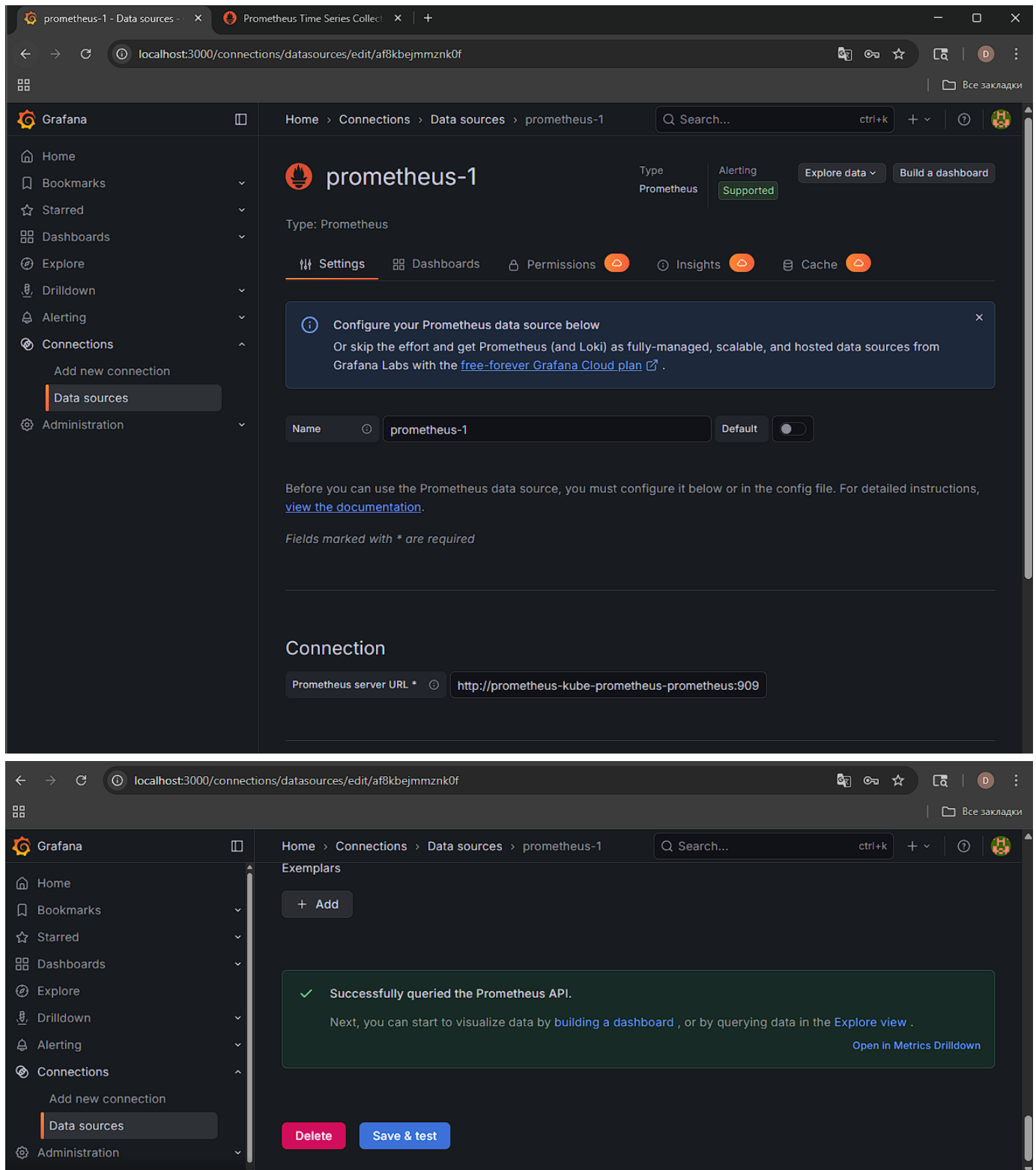
- Prometheus: <http://localhost:9090>
- Grafana: <http://localhost:3000> (логин `admin`, пароль `admin123`)



6.4. Настройка Grafana и дашборд

6.4.1. Data Source Prometheus

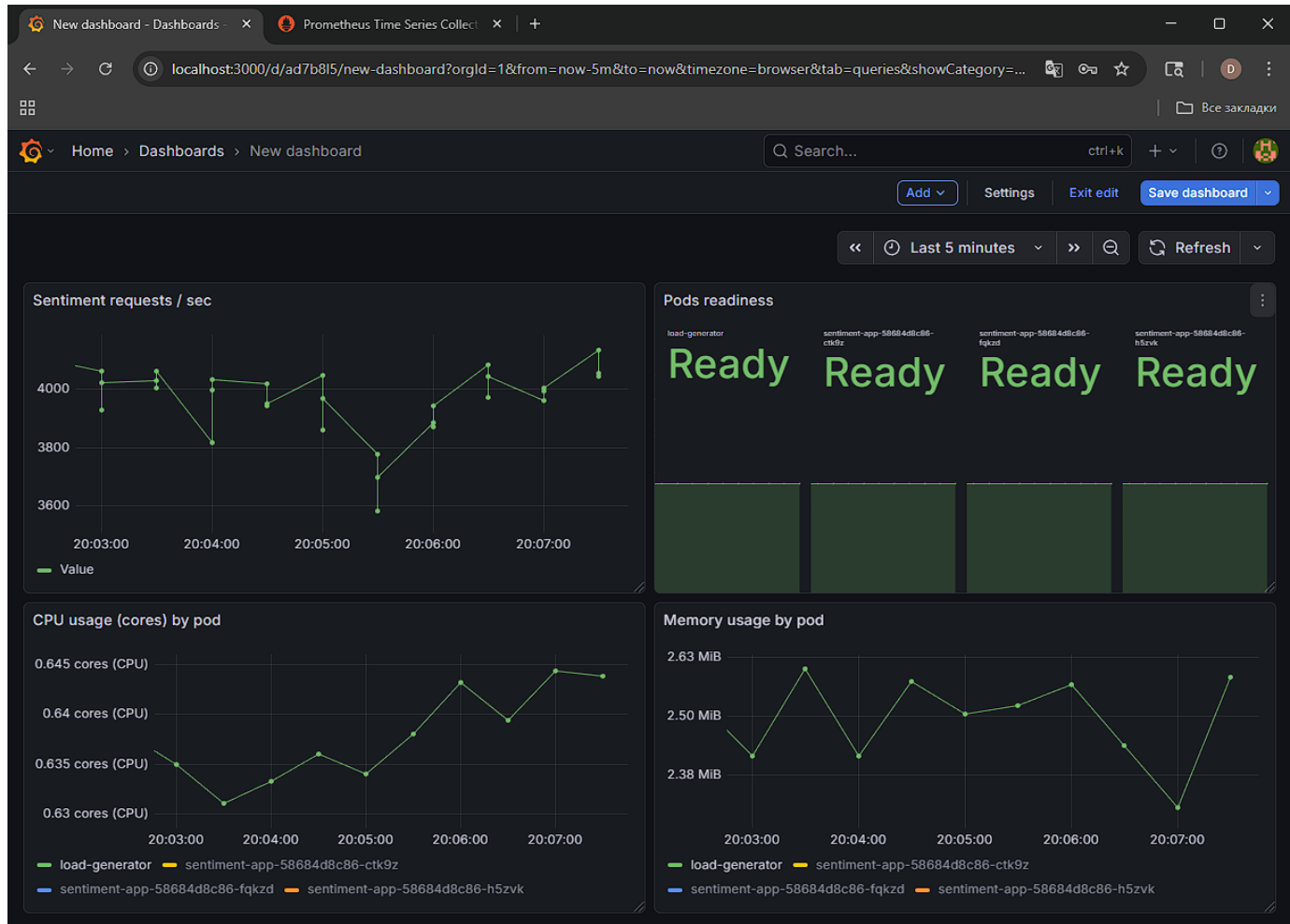
1. [Connections](#) → [Data sources](#) → [Add data source](#)
2. Prometheus
3. URL: <http://prometheus-kube-prometheus-prometheus:9090>
4. [Save & Test](#)



6.4.2. Дашборд

Панели:

- график `sentiment_requests_total` (по времени)
- графики `container_cpu_usage_seconds_total`, `container_memory_usage_bytes` (CPU/Memory по pod'ам)
- панель статусов pod'ов



7. Анализ тенденций (arxiv.org, 2024–2025 гг.)

7.1. Выбор литературы

Для анализа актуальных трендов были отобраны работы на arXiv за **2024–2025 гг.** по темам:

- **AI-подходы в оркестрации** (scheduling / rescheduling) и **автоскейлинге**;
- **serverless-исполнение** и борьба с **cold-start**;
- **инференс LLM** и инфраструктура вокруг него (маршрутизация, кэширование, GPU-гетерогенность);
- связь с Kubernetes/контейнерами (включая HPA, метрики, симуляторы, политики).

7.2. Сводная таблица выбранных исследованных работ

№	Название	Авторы	Год	Тематика	Ключевые результаты
1	Streamlining Resilient Kubernetes Autoscaling with Multi-Agent Systems via an Automated Online Design Framework	J. Soulé и др.	2025	Multi-agent подход к HPA-автоскейлингу и устойчивости	Авторы предлагают HPA-MAS (Multi-Agent System): декомпозиция цели “устойчивость к сбоям” на под-цели (в т.ч. при DDoS), обучение агентов в digital twin по трассам кластера и перенос политики в реальный кластер. ([arXiv][1])

№	Название	Авторы	Год	Тематика	Ключевые результаты
2	<i>Multi-Level ML Based Burst-Aware Autoscaling for SLO Assurance and Cost Efficiency (BAScaler)</i>	C. Meng и др.	2024	Burst-aware автоскейлинг с ML + RL	BAScaler различает “периодические пики” и “реальные bursts”, при bursts заранее завышает прогноз и выделяет ресурсы, а вне bursts использует reinforcement learning для коррекции оценок. В экспериментах: снижение SLO-нарушений и экономия ресурсов относительно базовых подходов. ([arXiv][2])
3	<i>AlBrix: Towards Scalable, Cost-Effective Large Language Model Inference Infrastructure</i>	AlBrix Team (J. Shan и др.)	2025	Cloud-native инфраструктура инференса LLM (Kubernetes-native)	Платформа для LLM-инференса с LLM-специфичным autoscaling , prefix/load-aware routing, distributed KV cache; гибридная оркестрация: Kubernetes (крупнозернистая) + Ray (тонкая). Отмечены улучшения throughput/latency за счёт кэширования и оркестрации. ([arXiv][3])
4	<i>KIS-S: A GPU-Aware Kubernetes Inference Simulator with RL-Based Auto-Scaling</i>	G. Zhang и др.	2025	GPU-инференс в Kubernetes: симулятор + RL-автоскейлер	KIS-S объединяет GPU-aware симулятор (KISim) и PPO-автоскейлер (KIScaler), который учится в симуляции и затем разворачивается без переобучения. Упор на GPU-метрики и bursty-нагрузку, где CPU-ориентированный HPA часто неадекватен. ([arXiv][4])
5	<i>Taming Cold Starts: Proactive Serverless Scheduling with Model Predictive Control</i>	C. Nguyen и др.	2025	Serverless scheduling: proactive mitigation cold-start	MPC-подход (Model Predictive Control) совместно оптимизирует prewarming контейнеров и dispatch запросов по прогнозу будущих вызовов. Реализация на Apache OpenWhisk в Kubernetes-тестбед; заявлено существенное снижение tail-latency и накладных расходов. ([arXiv][5])

7.3. Детальный анализ ключевых тенденций

7.3.1. От “порогов” к адаптивному AI-автоскейлингу: bursts, устойчивость, multi-agent

BAScaler (2024) показывает практический вектор развития HPA-подобных механизмов:

- проблема не в самом “масштабировании”, а в том, что нагрузка **неровная** (bursty), а SLO (Service Level Objective) — жёсткий;
- поэтому важны **распознавание bursts** и разная стратегия “burst vs non-burst”, плюс RL-коррекция ошибок оценивания. ([arXiv][2])

HPA-MAS (2025) развивает тему уже не как “одна умная политика”, а как **набор кооперативных агентов**, каждый из которых оптимизирует свою под-цель устойчивости (например, при деградациях/крашах/атакующих сценариях). Важный инженерный момент: авторы прямо закладывают **digital twin по трассам кластера** и перенос обученных политик в реальную среду. ([arXiv][1])

Связь с вашим стендом, Инженер Берёза. Если сейчас в учебном Kubernetes используется классический HPA на метриках уровня CPU/Memory, то логичный “шаг в будущее” (даже без полноценного ML в проде) — сделать инфраструктуру, совместимую с такими подходами:

- собирать **кластерные трассы** (Prometheus) и хранить их как датасет;
- перейти к HPA v2 / external metrics: latency, RPS, error-rate;
- тестировать политики сначала на “песочнице”/симуляции, затем переносить.

7.3.2. LLM-инференс в Kubernetes: autoscaling, маршрутизация, кэширование и GPU-метрики

AlBrix (2025) фиксирует важный тренд: в LLM-сценариях “просто поднять pods и включить HPA” недостаточно. Нужны:

- LLM-специфичные политики масштабирования (в т.ч. сглаживание метрик и быстрые реакции),
- умная маршрутизация (учёт префикса, кэша, стоимости вычислений),
- системные оптимизации (distributed KV cache),
- и даже гибридная оркестрация Kubernetes + Ray. ([arXiv][3])

KIS-S (2025) подчеркивает, почему CPU-ориентированные метрики ломаются на GPU-инференсе: bottleneck может быть в GPU-очередях/памяти/акселераторе, а не в CPU. Поэтому авторы строят GPU-aware симулятор и учат RL-автоскейлер в симуляции, затем разворачивают без retraining. ([arXiv][4])

Связь с проектом. Даже если текущее приложение не использует LLM/GPU, архитектурно это полезно как “вектор развития”:

- сервисы уже контейнеризованы → можно добавлять отдельный inference-service;
 - мониторинг Prometheus/Grafana → можно расширять метрики под latency/token-throughput;
 - в будущем возможна миграция в KServe/Knative-подобные паттерны (serverless inference) и внедрение cache-aware routing идей.
-

7.3.3. Serverless и cold-start: переход к proactive scheduling

Taming Cold Starts (2025) показывает тренд “управления будущим”: вместо реактивного масштабирования — **прогноз + оптимизация**. MPC-контроллер по прогнозу вызовов:

- решает, сколько контейнеров прогреть заранее,
- и как диспетчеризовать запросы, чтобы уменьшить cold-start без избыточных ресурсов. ([arXiv][5])

Связь с проектом. Если рассматривать ваш Minikube-стенд как модель “малого продакшена”, то именно cold-start-класс задач хорошо демонстрируется даже в учебной среде:

- легко воспроизвести bursty-нагрузку (тест-генератор),
 - измерить p95/p99 latency,
 - сравнить “reactive HPA” vs “prewarming/прогноз”.
-

7.4. Итоговые тенденции и рекомендации

Основные тенденции (2024–2025):

1. **Интеллектуальный автоскейлинг:** bursts + SLO-ориентация + RL-коррекция вместо “одного порога”. ([arXiv][2])
2. **Устойчивость как цель оркестрации:** multi-agent подходы, цифровые двойники по трассам, перенос политик в реальный кластер. ([arXiv][1])
3. **LLM-инференс требует специализированной инфраструктуры** (маршрутизация, кэш, autoscaling “по-умному”, гибридная оркестрация). ([arXiv][3])
4. **GPU-aware управление ресурсами:** метрики и политики должны учитывать GPU-узкие места, а симуляция становится ключевым инструментом разработки автоскейлеров. ([arXiv][4])
5. **Proactive serverless scheduling:** прогнозирование + управление prewarming для снижения cold-start и tail-latency. ([arXiv][5])

Практические рекомендации для нашего стенда:

- Зафиксировать текущий HPA как baseline и добавить экспериментальный контур: **external metrics** (latency/RPS/error-rate) → это подготовит почву для AI-подходов.
- Начать собирать “трассы нагрузки” как датасет (Prometheus) и описать это как шаг к **digital twin**, по аналогии с работой про HPA-MAS.

- Отдельно отметить “вектор развития”: LLM-инференс в Kubernetes (AlBrix-подход) и GPU-aware autoscaling (KIS-S) — как дальнейшее расширение архитектуры без переделки базовой контейнерной части.
- В качестве демонстрационного исследования добавить сценарий “bursty-нагрузки и cold-start”, где сравнить реактивный подход и prewarming-идею (по мотивам MPC-scheduler).

8. Заключение

8.1. Достигнутые результаты

- Подготовлено и запущено Java-приложение, реализующее:
 - `GET /api/sentiment?text=...` — обработка запроса и выдача результата
 - `GET /health` — endpoint проверки состояния сервиса
 - `GET /metrics` — экспорт метрик в формате Prometheus
- Выполнена контейнеризация приложения:
 - создан Dockerfile и собран Docker-образ приложения
 - выполнена локальная проверка запуска контейнера и работоспособности API
- Развёрнут Kubernetes-кластер в Minikube и подготовлена базовая инфраструктура:
 - включены необходимые компоненты кластера (Ingress, сбор метрик для autoscaling)
 - выполнена проверка состояния кластера (`kubectl get nodes/pods`)
- Приложение развернуто в Kubernetes:
 - `Deployment` приложения и запуск нескольких реплик
 - `Service` для доступа к приложению внутри кластера
 - `Ingress` для маршрутизации входящего трафика
 - `HPA` (Horizontal Pod Autoscaler) — подтверждена работа масштабирования по CPU под нагрузкой
- Настроен мониторинг и наблюдаемость:
 - установлен стек Prometheus + Grafana (kube-prometheus-stack)
 - подключен Prometheus datasource в Grafana и проверено получение метрик
 - создан Grafana-дашборд с панелями:
 - **Sentiment requests / sec**
 - **CPU usage (cores) by pod**
 - **Memory usage by pod**
 - **Pods readiness**
- Выполнен анализ тенденций по статьям arXiv (2024–2025).

8.2. Трудности и решения

8.2. Трудности и решения

Трудность	Описание	Решение
Установка Docker в Ubuntu VM	На этапе установки возникали расхождения в ожидаемом/фактическом выводе команд и сомнения в корректности установки.	Установил Docker и проверил работоспособность через базовые команды (<code>docker version</code> , <code>docker ps</code>). Работу выполнял через SSH из VSCode — это не влияет на результат, если команды выполняются на целевой VM.
Ограничение ресурсов VM/Minikube	Был риск, что выделенных ресурсов (память/CPU/диск) не хватит для мониторинга и приложения (типичный симптом — Pending/OOM/нестабильный старт).	Оставил текущие ресурсы, т.к. кластер и мониторинг запустились. Зафиксировал правило: при Pending/OOM — увеличивать RAM/CPU/диск VM и перезапускать Minikube.

Трудность	Описание	Решение
Включение необходимых аддонов Minikube	Без необходимых аддонов часть функциональности кластера не проверялась (Ingress/HPA/метрики).	Включил требуемые аддоны (в первую очередь <code>ingress</code> , <code>metrics-server</code>), после чего Ingress и HPA стали работоспособны и проверяемы.
Grafana/Prometheus: проверка источника данных	Требовалось убедиться, что Grafana действительно получает данные из Prometheus.	Настроил Prometheus datasource в Grafana, выполнил <code>Save & Test</code> , проверил метрики через Explore (например, <code>up</code>).
Grafana “No data” в панели CPU	Запросы по CPU не возвращали данные из-за несовпадения метрик/лейблов и неверной фильтрации.	Через Prometheus UI проверил наличие метрики и реальные labels, после чего использовал рабочий PromQL для CPU по pod’ам: <code>sum by (pod) (rate(container_cpu_usage_seconds_total{cpu="total", pod=~"sentiment-app-.* load-generator"}[1m]))</code> .
Grafana “No data” в панели Memory (первый вариант)	Простой запрос по памяти не попадал в нужные series/pod’ы.	Перешёл на <code>container_memory_working_set_bytes</code> и добавил корректные фильтры: <code>sum by (pod) (container_memory_working_set_bytes{namespace="default", pod=~"sentiment-app-.* load-generator", id=~"/kubepods.*"}).</code>
Некорректное отображение readiness (“нет time field”)	Панель readiness отображалась неверно из-за неподходящего типа данных/агрегации по времени.	Использовал метрику kube-state-metrics и time-агрегацию: <code>avg_over_time(kube_pod_status_ready{namespace="default", condition="true"}[1m])</code> , после чего статусы начали отображаться корректно.
Метрики “не меняются” без нагрузки	Без генерации запросов метрика <code>sentiment_requests_total</code> почти не менялась, графики выглядели статичными.	Запустил <code>load-generator</code> , который циклически отправляет запросы к сервису — метрики стали динамичными, появилась нагрузка для наблюдения поведения HPA.
Ошибка <code>AlreadyExists</code> при повторном запуске <code>load-generator</code>	При повторном создании pod с тем же именем Kubernetes возвращал <code>AlreadyExists</code> .	Удалял существующий pod перед повторным запуском (<code>kubectl delete pod load-generator</code>) либо запускал под другим именем.

8.3. Перспективы развития

- Перейти от rule-based логики к ML/LLM-подходу: обучаемая модель тональности (эмбединги, fine-tuning или внешнее API), а также добавление метрик качества (precision/recall/F1) и тестового датасета для регрессионной проверки.
- Повысить производительность и устойчивость: кэширование результатов `/api/sentiment` (например, Redis) + ограничение частоты запросов (rate limiting) и базовая защита от перегрузки.
- Автоматизировать поставку: CI/CD (GitHub Actions / GitLab CI) — сборка и проверка (tests/lint), публикация Docker-образа в registry, деплой в Kubernetes по тегам/релизам.
- Улучшить наблюдаемость: распределённый трейсинг (Jaeger или Grafana Tempo) и сквозная корреляция логов/метрик через `traceId` (единый идентификатор трассировки).
- Усилить сетевую безопасность и контроль трафика: внедрение service mesh (Istio или Linkerd) для mTLS (mutual TLS — взаимная аутентификация), политик доступа, ретраев/таймаутов и продвинутой телеметрии.
- Ввести эксплуатационные практики: алёрты (Alertmanager), SLO/SLI (Service Level Objective / Service Level Indicator — целевые уровни сервиса и измеримые показатели), отдельный дашборд “Golden Signals” (latency/traffic/errors/saturation) и сценарии реагирования (runbooks).

9. Список использованных источников

9.1. Документация

1. Minikube: <https://minikube.sigs.k8s.io/docs/start/>
2. Kubernetes Docs: <https://kubernetes.io/docs/home/>
3. Helm: <https://helm.sh/docs/>
4. kube-prometheus-stack: <https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack>
5. Grafana: <https://grafana.com/docs/>
6. Prometheus (PromQL): <https://prometheus.io/docs/prometheus/latest/querying/basics/>
7. Kubernetes HPA: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>

9.2. Научные работы (arXiv, 2024–2025)

1. Streamlining Resilient Kubernetes Autoscaling with Multi-Agent Systems via an Automated Online Design Framework: <https://arxiv.org/abs/2505.21559>
 2. Multi-Level ML Based Burst-Aware Autoscaling for SLO Assurance and Cost Efficiency: <https://arxiv.org/abs/2402.12962>
 3. AlBrix: Towards Scalable, Cost-Effective Large Language Model Inference Infrastructure: <https://arxiv.org/abs/2504.03648>
 4. KIS-S: A GPU-Aware Kubernetes Inference Simulator with RL-Based Auto-Scaling: <https://arxiv.org/abs/2507.07932>
 5. Taming Cold Starts: Proactive Serverless Scheduling with Model Predictive Control: <https://arxiv.org/abs/2508.07640>
-